

Hidden Markov models

A hidden Markov chain emits what we see; three passes over one trellis answer the usual questions.

THE QUESTION

We see the tosses H, H, T, H but never the coin. How likely is the sequence, which coin made each toss, and how do we learn the coins?

THE INTUITION

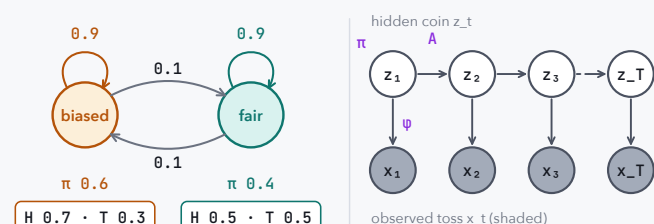
The coin follows a Markov chain; each toss depends only on the current coin. Summing over all K^T coin paths is hopeless, but they all run through one $K \times T$ trellis, so dynamic programming shares the work.

KEY TAKEAWAY

Forward α : likelihood and filter. Backward β : smoothing. Max for sum: the best path (Viterbi). With labels, learn by counting; without, EM counts in expectation.

$\pi, A, \varphi \rightarrow$ forward $\alpha \rightarrow L = \sum \alpha_T \rightarrow$ backward $\beta \rightarrow$ smooth $\alpha \beta \rightarrow$ Viterbi: max $\rightarrow$ EM: expected counts

1 Hidden states emit what we see



The coin $z_t \in \{1, \dots, K\}$ is a first-order Markov chain; the toss x_t depends only on z_t . Parameters $\theta = \{\pi, A, \varphi\}$:

$$\pi_k = p(z_1 = k), \quad A_{jk} = p(z_t = k \mid z_{t-1} = j),$$

and emissions $\varphi_j(x) = p(x_t = x \mid z_t = j)$. Rows of A sum to 1 and A is the same at every t (stationarity). φ is a table for coins, or one density per state for continuous data (the slides' appliance: ON $\mathcal{N}(100, 10)$, OFF $\mathcal{N}(0, 5)$ watts). The joint factorises:

$$p(x_{1:T}, z_{1:T}) = \pi_{z_1} \varphi_{z_1}(x_1) \prod_{t=2}^T A_{z_{t-1}z_t} \varphi_{z_t}(x_t)$$

To sample, draw $z_1 \sim \pi$, $x_1 \sim \varphi_{z_1}$, then each z_t from row z_{t-1} of A . With the states visible it is a plain Markov chain (see the MCMC sheet): the slides' weather chain gives $P(\text{R R S S C}) = 0.2 \cdot 0.4 \cdot 0.3 \cdot 0.8 \cdot 0.1 = 0.00192$.

2 The likelihood sums over every hidden path

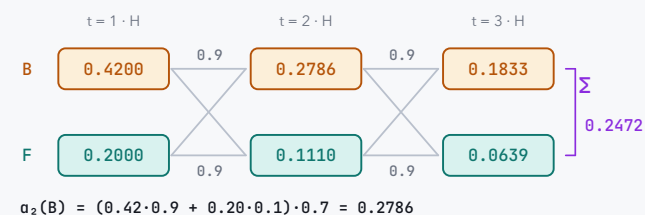
$z_1 z_2$	$\pi \cdot \varphi(H) \cdot A \cdot \varphi(H)$	$p(H, H, z_1, z_2)$
B B	$0.6 \cdot 0.7 \cdot 0.9 \cdot 0.7$	0.2646
F B	$0.4 \cdot 0.5 \cdot 0.1 \cdot 0.7$	0.0140
B F	$0.6 \cdot 0.7 \cdot 0.1 \cdot 0.5$	0.0210
F F	$0.4 \cdot 0.5 \cdot 0.9 \cdot 0.5$	0.0900

sum over 2^2 paths $L(H, H) = 0.3896$ $K^2 T$ paths: $2^2 100 = 1.3e30$

$$p(x_{1:T} \mid \theta) = \sum_{z_{1:T}} p(x_{1:T}, z_{1:T} \mid \theta)$$

Each path is a product of $2T$ factors and there are K^T paths, so brute force costs $O(TK^T)$: 4 products for two tosses, 1.3×10^{30} for 100. Paths that share a prefix repeat the same partial products; the forward pass computes each one once.

3 The forward pass shares the sums



$\alpha_t(i) = p(x_{1:t}, z_t = i)$: the tosses so far and the coin now. Every path into node $(t+1, j)$ passes through some node at t , so

$$\alpha_1(i) = \pi_i \varphi_i(x_1), \quad \alpha_{t+1}(j) = \left[\sum_i \alpha_t(i) A_{ij} \right] \varphi_j(x_{t+1})$$

and $p(x_{1:T}) = \sum_i \alpha_T(i)$, here equal to the sum of all 8 path products. Each step costs K^2 products, $O(K^2 T)$ in all.

4 Filtering normalises α

H	biased 0.677	0.323	0.42 / 0.62
H H	biased 0.715	0.285	slides: fair 0.289
H H H	biased 0.742	0.258	0.1833 / 0.2472

$p(z_t = i \mid x_{1:t}) = \alpha_t(i) / \sum_j \alpha_t(j)$, online, one toss at a time. After H: $0.42/0.62 = 0.677$ (the slides round to 2/3). After H, H: biased 0.715, fair $0.1110/0.3896 = 0.285$. Each head moves belief toward the biased coin.

5 Scale each column, keep the logs

For 2000 seeded tosses $L \approx 10^{-581}$: 0 in double precision. Normalise each column (the filter) and add up the logs of the normalisers:

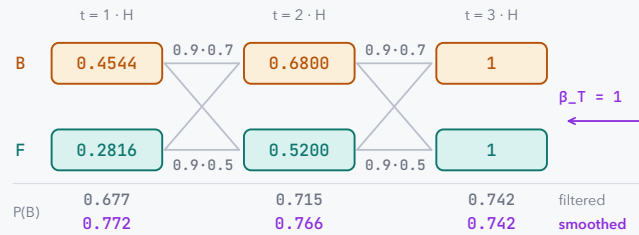
```
def filt_loglik(o, pi, A, E):
    # E[state, symbol]; o = symbol ids
    a, ll = pi * E[:, o[0]], 0.0
    for t, s in enumerate(o):
        if t: a = (a @ A) * E[:, s]
        c = a.sum(); a = a / c
        ll += np.log(c)
    return a, ll # p(z_T | x), log L
```

Viterbi scales the same way: add $\log A$ and $\log \varphi$, then take the max.

Smooth, decode, then learn

A backward pass adds the future, two notions of "best" pick states, and counts (or expected counts) give π , A and φ .

6 The backward pass adds the future

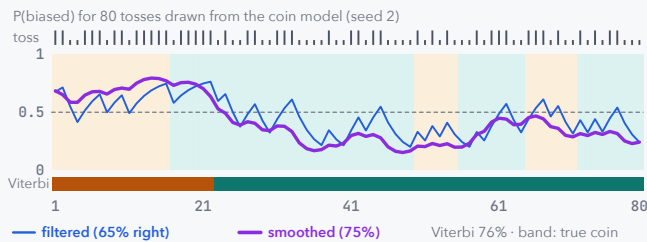


$\beta_t(i) = p(x_{t+1:T} | z_t = i)$, with $\beta_T(i) = 1$ and $\beta_t(i) = \sum_j A_{ij} \varphi_j(x_{t+1}) \beta_{t+1}(j)$. Given z_t the future is independent of the past, so

$$\gamma_t(i) = p(z_t = i | x_{1:T}) = \frac{\alpha_t(i) \beta_t(i)}{\sum_j \alpha_t(j) \beta_t(j)}.$$

For H, H, H: $\gamma_1(B) = 0.42 \cdot 0.4544 / 0.2472 = 0.772$, up from 0.677; with only H, H it is 0.733. The denominator equals L at every t : a free check.

7 Smoothing uses the whole sequence



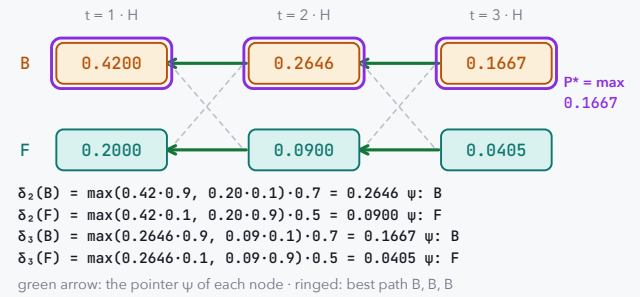
The filter sees only the past and reacts late to each switch; the smoothed curve also sees later tosses. With $P(H)$ 0.7 against 0.5 the coins are hard to tell apart: over 2000 seeded runs of 80 tosses, 63.6% of tosses are decoded right by the filter, 66.5% by smoothing, 61.9% by Viterbi.

8 Most probable states or most probable path

	$z_1 = 1$	$z_1 = 2$	
$z_2 = 1$	0.04	0.30	0.34
$z_2 = 2$	0.36	0.30	0.66
sums	0.40	0.60	

MPM takes $\hat{z}_t = \arg \max_i \gamma_t(i)$ at each step; MAP takes $z^* = \arg \max_{z_{1:T}} p(z_{1:T} | x_{1:T})$. In the slides' two-step table MPM gives (2, 2), joint 0.30, not the best pair. MPM maximises the expected number of correct steps (hence its lead above); MAP returns one consistent path. Smoothing gives MPM; Viterbi (next panel) gives MAP. For H, H, H both give B, B, B.

9 Viterbi: max in place of sum, plus pointers



$\delta_t(i)$ is the best single path ending in $z_t = i$ that explains $x_{1:t}$:

$$\delta_1(i) = \pi_i \varphi_i(x_1), \quad \delta_{t+1}(j) = \left[\max_i \delta_t(i) A_{ij} \right] \varphi_j(x_{t+1})$$

and $\psi_{t+1}(j)$ stores the arg max. Take $z_T^* = \arg \max_i \delta_T(i)$, then $z_t^* = \psi_{t+1}(z_{t+1}^*)$. Here $p(B, B, B | x) = 0.1667 / 0.2472 = 0.674$; next best F, F, F at 0.0405. Also $O(K^2 T)$; use logs on long runs.

10 Labelled sequences: learning is counting

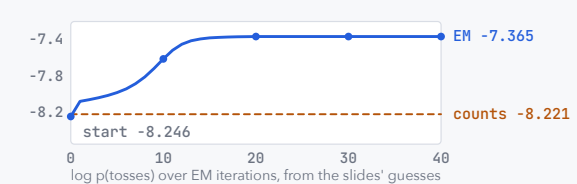
coin (colour), toss	starts	$\pi = (2/3, 1/3)$
s1: H T H H	B → B 3 B → F 1	$A_B = (0.75, 0.25)$
s2: H T T T	F → B 1 F → F 4	$A_F = (0.20, 0.80)$
s3: T H H H	B: 5 H in 6	$\varphi_B(H) = 5/6$
	F: 2 H in 6	$\varphi_F(H) = 1/3$

With z observed the log-likelihood splits into separate sums and the MLE is a ratio of counts:

$$\hat{\pi}_j = \frac{N_j^1}{N}, \quad \hat{A}_{jk} = \frac{N_{jk}}{\sum_k N_{jk}}, \quad \hat{\varphi}_{jl} = \frac{N_{jl}}{N_j}$$

over N sequences: starts in j , $j \rightarrow k$ transitions, symbol l seen in state j . A Gaussian φ takes the mean and covariance of each state's x .

11 No labels: EM counts in expectation



E-step: from α, β get $\gamma_t(i)$ and $\xi_t(i, j) = \alpha_t(i) A_{ij} \varphi_j(x_{t+1}) \beta_{t+1}(j) / L$. M-step: the counting formulas with $\sum \gamma, \sum \xi$ as counts. $\log L$ never drops. From the slides' guesses it reaches -7.365 , above the counting estimates, by making "fair" a tails-only coin. 50 random starts end between -8.15 and -7.365 , so restart, or start from labels or Viterbi training.

α : sum over the past · β : sum over the future · $\gamma \propto \alpha\beta$: smoothed state · $\delta + \psi$: best path · counts or expected counts give π, A, φ

Source: lecture slides "Hidden Markov models" (handwritten), nipunbatra.github.io/pml-teaching. Numbers recomputed in NumPy (forward, backward and Viterbi checked by enumerating every hidden path; seeded samples). Corrections: after H, H the filtered fair value is 0.285, not 0.289; the three labelled sequences give $\varphi_B(H) = 5/6$ and $\varphi_F(H) = 1/3$, not 0.8 and 0.4; in the EM example $\beta_2(B) = 0.4432$, not 0.4192, since $\beta_3(F) = 0.52$, not $1 - \beta_3(B)$.